# Numerically stable gradients of Golub-Kahan Bidiagonalization

Theo Würtzen and Nicholas Krämer

September 12, 2025

## Contents

**1 Rambling:** **2**

**2 Background** **2**
2.1 Notation . . . . . 2
2.2 Matrix free linear algebra . . . . . 2
2.3 Bidiagonalization . . . . . 3
2.4 Hessenberg Factorization . . . . . 3
2.5 Hessenberg factorization generalizes Bidiagonalization . . . . . 4

**3 Adjoint systems** **5**
3.1 Adjoint system of Hessenberg Factorization . . . . . 5
3.2 Solving the Adjoint System . . . . . 5
3.3 Derivation of the Stable Adjoint System . . . . . 8
3.3.1 Step 3: Profit . . . . . 9
3.4 Columnwise expanded unstable adjoint . . . . . 10
3.5 Hessenberg factorization has stable adjoint because of reprojection . . . . . 10
3.6 Unstable Primal constraints . . . . . 10
3.7 Unstable Adjoint system . . . . . 11
3.8 Stable Adjoint System . . . . . 11
3.9 The Adjoint Method to compute gradients . . . . . 11

**4 Numerical Stability and Experiments** **13**
4.1 Reprojection and Reorthonormalization . . . . . 13

# 1 Rambling:

Bidiagonalization suffers from loss of orthogonality of the vectors in $L$ and $R$. This can be solved by reorthonormalizing the new vector against all computed vectors.

Ideally, that should've been enough, but it can be shown that it is not - we need at least 3 applications of reorthonormalization for this to work. [1].

In the adjoint system we can do something similar.

UPARROW IS PHI, DOWNARROW IS XI

# 2 Background

While Bidiagonalization has well-understood forward algorithms and constraints, computing its gradient in a numerically stable way is challenging (TODO show that it is unstable from example). This paper builds from a connection between Bidiagonalization and Hessenberg factorization, namely that any Bidiagonalization can be computed by performing Hessenberg factorization on an appropriately constructed symmetric matrix, allowing us to then leverage the stable gradient of Hessenberg factorization proposed in (TODO cite Nico).

## 2.1 Notation

Let $e_j$ be the $j$th unit vector. Denote by "$\circ$" the element-wise matrix product which binds weaker than the standard matrix product and stronger than addition, and define the matrices

$$\mathbf{1} := [1]_{i,j=1}^{K} \qquad I_{\leq} := [\delta_{i\leq j}]_{i,j=1}^{K} \qquad I_{<} := [\delta_{i<j}]_{i,j=1}^{K} \qquad I_{\ll} := [\delta_{i+1<j}]_{i,j=1}^{K}$$

so that for example, $\mathbf{1} \circ A = A$, $I_{\leq} \circ A$ extracts the lower triangular matrix of $A$ (including the diagonal), and $I_{\ll} \circ A = 0$ enforces Hessenberg form.

## 2.2 Matrix free linear algebra

Matrix-free linear algebra refers to implementing matrix operations without explicitly forming or storing the full matrices. Instead, we work with functions that compute matrix-vector products. This is particularly useful when dealing with large matrices where storing them would be impractical, or when the matrix is defined implicitly through some operation, where we never need the full matrix, only specific matrix-vector products. For a linear operator $A : \mathbb{R}^m \to \mathbb{R}^n$, instead of storing an $n \times m$ matrix, we represent it as a function:

$$\text{matvec} : \mathbb{R}^m \times \Theta \to \mathbb{R}^n$$

[1] Show this in an experiment

such that $\text{matvec}(v, \theta) = A_\theta v$ for any vector $v \in \mathbb{R}^m$ and parameters $\theta \in \Theta$. This approach enables a general way to handle general linear operators, such as matrix-vector products with sparse and structured matrices, the jacobian-vector product of a neural network, finite-difference operators, discrete fourier transforms and other linear operators acting between vector spaces.

## 2.3 Bidiagonalization

$$\texttt{Bidiagonalize}_K : (A, v) \mapsto (L, R, B, r, c)$$

Bidiagonalizing an arbitrary matrix $A \in \mathbb{R}^{N \times M}$ with initial vector $v \in \mathbb{R}^M$ and a prescribed number of iterations $K \in \mathbb{N}_+$, will yield column-orthonormal $L \in \mathbb{R}^{N \times K}$ and $R \in \mathbb{R}^{M \times K}$, bidiagonal $B \in \mathbb{R}^{K \times K}$, residual vector $r \in \mathbb{R}^N$ and inverse length $c \in \mathbb{R}$, satisfying the following constraints:

$$A^\top L = RB^\top + r(e_k)^\top \tag{1}$$
$$AR = LB \tag{2}$$
$$Re_1 = cv \tag{3}$$
$$I_{\leq} \circ [L^\top L] = I \tag{4}$$
$$I_{\leq} \circ [R^\top R] = I \tag{5}$$
$$R^\top r = 0 \tag{6}$$

and $B$ is diagonally banded as follows:

$$B = \underbrace{\text{diag}(\boldsymbol{\alpha}) + \text{diag}(\boldsymbol{\beta}, \text{k=1})}_{\text{B is 'upper bidiagonal'}} = \begin{bmatrix} \alpha_1 & \beta_1 & 0 & 0 & 0 & 0 \\ 0 & \alpha_2 & \beta_2 & 0 & 0 & 0 \\ 0 & 0 & \ddots & \ddots & 0 & 0 \\ 0 & 0 & 0 & 0 & \alpha_{K-1} & \beta_{K-1} \\ 0 & 0 & 0 & 0 & 0 & \alpha_K \end{bmatrix} \in \mathbb{R}^{K \times K} \tag{7}$$

## 2.4 Hessenberg Factorization

$$\texttt{Hessenberg}_K : (A, v) \mapsto (Q, H, r, c)$$

Given a square matrix $A \in \mathbb{R}^{N \times N}$ an initial vector $v \in \mathbb{R}^N$ and a prescribed number of iterations $K \in \mathbb{N}_+$, the Hessenberg factorization produces a column-orthogonal $Q \in \mathbb{R}^{N \times K}$, structured $H \in \mathbb{R}^{K \times K}$, residual vector $r \in \mathbb{R}^N$, and reciprocal length $c \in \mathbb{R}$ such that

$$AQ = QH + r(e_K)^\top \tag{8}$$
$$Qe_1 = cv \tag{9}$$
$$I_{\leq} \circ [Q^\top Q] = I \tag{10}$$
$$I_{\ll} \circ H = 0 \tag{11}$$
$$Q^\top r = 0 \tag{12}$$

## 2.5 Hessenberg factorization generalizes Bidiagonalization

To compute $\texttt{Bidiagonalize}_K(A, v) = (L, R, B, r, c)$ for any $A \in \mathbb{R}^{N\times M}$ and $v \in \mathbb{R}^M$ using Hessenberg factorization, we construct the symmetric matrix $\dddot{A}$ and augmented vector $\ddot{v}$:

$$\dddot{A} := \begin{bmatrix} \mathbf{0} & A \\ A^\top & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{(N+M)\times(N+M)} \quad \text{and} \quad \ddot{v} := \begin{pmatrix} \mathbf{0} \\ v \end{pmatrix} \in \mathbb{R}^{N+M}$$

We define the operation $\texttt{augment} : (A, v) \mapsto (\dddot{A},\ \ddot{v})$ as the augmentation of a matrix and vector to this symmetric matrix and padded vector. Applying Hessenberg factorization for $2K$ iterations to this augmented input yields $\texttt{Hessenberg}_{2K}(\dddot{A},\ \ddot{v}) = (\ddot{Q}, \dddot{H},\ \ddot{r}, c)$ which are directly related to $(L, R, B, r, c)$ through the following sparse equations:

$$\ddot{Q} = \begin{bmatrix} \mathbf{0} & l_1 & \mathbf{0} & l_2 & \cdots & \mathbf{0} & l_k \\ r_1 & \mathbf{0} & r_2 & \mathbf{0} & \cdots & r_k & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{(N+M)\times 2K} \qquad \ddot{r} := \begin{pmatrix} \mathbf{0} \\ r \end{pmatrix} \in \mathbb{R}^{N+M}$$

$$\dddot{H} = \begin{bmatrix} 0 & \alpha_1 & 0 & 0 & 0 & 0 & 0 & 0 \\ \alpha_1 & 0 & \beta_1 & 0 & 0 & 0 & 0 & 0 \\ 0 & \beta_1 & 0 & \alpha_2 & 0 & 0 & 0 & 0 \\ 0 & 0 & \alpha_2 & 0 & \beta_2 & 0 & 0 & 0 \\ 0 & 0 & 0 & \beta_2 & 0 & \ddots & 0 & 0 \\ 0 & 0 & 0 & 0 & \ddots & 0 & \beta_{K-1} & 0 \\ 0 & 0 & 0 & 0 & 0 & \beta_{K-1} & 0 & \alpha_K \\ 0 & 0 & 0 & 0 & 0 & 0 & \alpha_K & 0 \end{bmatrix} \in \mathbb{R}^{2K\times 2K}$$

The vectors $l_i$ and $r_i$ in $\ddot{Q}$ are exactly the columns of $L$ and $R$, while the bands in $\dddot{H}$ hold the entries of the bidiagonal matrix $B$ with duplicated $\alpha_i$ and $\beta_i$ values. The residual vector $\ddot{r}$ is simply a padded version of $r$, and the inverse norm $c$ is unchanged.
We formalize this relationship through a simple linear extraction mapping

$$\texttt{extract} : (\ddot{Q}, \dddot{H},\ \ddot{r}, c) \mapsto (L, R, B, r, c)$$

that projects away the zeros and extracts the relevant non-zero entries from the Hessenberg outputs.[2] This allows us to theoretically define **augmented Bidiagonalization** as

$$\overline{\texttt{Bidiagonalize}}_K := \texttt{extract} \circ \texttt{Hessenberg}_{2K} \circ \texttt{augment} : (A, v) \mapsto (L, R, B, r, c)$$

which is functionally equivalent to standard Bidiagonalization but inherits the numerical stability properties of Hessenberg factorization's adjoint system.
The problem is that this is "computing a lot of zeros". We will show that we can prune the computational graph to minimize work.

[2]The projection from $\dddot{H}$ to $B$ is not unique, as $\dddot{H}$ contains two copies of each $\alpha_i$ and $\beta_i$. We arbitrarily select the superdiagonal entries and drop the subdiagonal entries. This choice affects the sparsity pattern of $\nabla_{\dddot{H}}\rho$ in section 3.3. Any other convex linear combination of each pair would also be valid.

# 3 Adjoint systems

## 3.1 Adjoint system of Hessenberg Factorization

Let $K \in \mathbb{N}_+$, $A \in \mathbb{R}^{N\times N}$, $v \in \mathbb{R}^N$, $H \in \mathbb{R}^{K\times K}$, $Q \in \mathbb{R}^{N\times K}$, $r \in \mathbb{R}^N$, $c \in \mathbb{R}$ satisfy the primal constraints (8) through (12). Let adjoint variables $\Lambda \in \mathbb{R}^{N\times K}$ with $\Lambda e_i = \lambda_i$, $\Gamma \in \mathbb{R}^{K\times K}$, $\lambda \in \mathbb{R}^N$, $\gamma \in \mathbb{R}^K$ solve the following system of equations:

$$0 = \nabla_Q \rho + A^\top \Lambda - \Lambda H^\top + \lambda (e_1)^\top + Q(I_\leq \circ \Gamma) + Q(I_\leq \circ \Gamma)^\top + r\gamma^\top \tag{13}$$
$$0 = \nabla_H \rho - Q^\top \Lambda + I_\ll \circ \Sigma \tag{14}$$
$$0 = \nabla_r \rho - \Lambda e_K + Q\gamma \tag{15}$$
$$0 = \nabla_c \rho - v^\top \lambda \tag{16}$$

The gradients $\nabla_A \rho$ and $\nabla_v \rho$ are then given by[3]:

$$\nabla_A \rho = \Lambda Q^\top \tag{17}$$
$$\nabla_v \rho = -c\lambda \tag{18}$$

## 3.2 Solving the Adjoint System

The output gradients $\nabla_Q \rho$, $\nabla_H \rho$, $\nabla_r \rho$, $\nabla_c \rho$ will have know quantities, so we need only determine the adjoint variables $\Lambda$, $\Gamma$, $\lambda$, $\gamma$. $\Gamma$ is involved only in equation (13) in which only the lower triangular part is accessed, so for convenience we will define $\Gamma$ to be lower triangular, meaning we can effectively drop the mask $\Gamma := I_\leq \circ \Gamma$.
We get expressions for $\lambda_K$ and $\gamma$:

$$(15) \iff \lambda_K = \nabla_r \rho + Q\gamma \tag{19}$$
$$\overset{(10)}{\implies} Q^\top \lambda_K = Q^\top \nabla_r \rho + Q^\top Q\gamma = Q^\top \nabla_r \rho + \gamma$$
$$\iff \gamma = Q^\top \lambda_K - Q^\top \nabla_r \rho \tag{20}$$

To evaluate $Q^\top \lambda_k$, inspect (14):

$$(14) \iff Q^\top \Lambda = \nabla_H \rho + I_\ll \circ \Sigma$$
$$\implies (\mathbf{1} - I_\ll) \circ Q^\top \Lambda = (\mathbf{1} - I_\ll) \circ \nabla_H \rho + (\mathbf{1} - I_\ll) \circ I_\ll \circ \Sigma = (\mathbf{1} - I_\ll) \circ \nabla_H \rho \tag{21}$$
$$\implies Q^\top \lambda_K = \nabla_H \rho e_K$$
$$\overset{(20)}{\implies} \gamma = \nabla_H \rho e_K - Q^\top \nabla_r \rho \tag{22}$$

[3] note a sign difference between (18) and the result in TODO CITE due to a typo

This final relation lets us express $\gamma$ in terms of the known quantities, which we can then plug into (19) to get $\lambda_K$.
Next, we will get expressions for the remaining columns of $\Lambda$ and $\Gamma$, as well as for $\lambda$. As a preliminary, from primal constraints (9) we have $v = q_1/c$, yielding

$$(16) \iff c\nabla_c \rho = q_1^\top \lambda \tag{23}$$

We now turn to adjoint system constraint (13), left-multiply $Q^\top$ and then apply a mask to consider only the upper triangular part of the equations:

$$\begin{aligned}
(13) \overset{(10),(12)}{\implies} \quad & \mathbf{0} = Q^\top \nabla_Q \rho + Q^\top A^\top \Lambda - Q^\top \Lambda H^\top + Q^\top \lambda (e_1)^\top + \Gamma + \Gamma^\top \\
\implies \quad & \mathbf{0} = I_{\geq} \circ \left[ Q^\top \nabla_Q \rho + Q^\top A^\top \Lambda - Q^\top \Lambda H^\top + Q^\top \lambda (e_1)^\top + \Gamma + \Gamma^\top \right] \\
& = I_{\geq} \circ \left[ Q^\top \nabla_Q \rho + Q^\top A^\top \Lambda - Q^\top \Lambda H^\top + Q^\top \lambda (e_1)^\top \right] + \operatorname{diag}(\Gamma) + \Gamma^\top
\end{aligned}$$

We now inspect column $i$ in this matrix equation. We will use sparsity of $H$ to rewrite the matrix product $Q^\top \Lambda H^\top$, and we will apply some simplifications with the mask $I_{\geq}$:

$$\begin{aligned}
\mathbf{0} &= \left( I_{\geq} \circ \left[ Q^\top \nabla_Q \rho + Q^\top A^\top \Lambda - Q^\top \Lambda H^\top + Q^\top \lambda (e_1)^\top \right] \right) e_i + \left( \operatorname{diag}(\Gamma) + \Gamma^\top \right) e_i \\
&= I_{\geq} e_i \circ \left[ Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i - Q^\top \Lambda H^\top e_i + Q^\top \lambda (e_1)^\top e_i \right] + \operatorname{diag}(\Gamma) e_i + \Gamma^\top e_i \\
&= I_{\geq} e_i \circ \left[ Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i - Q^\top \left( \sum_{j=i-1}^{K} \lambda_j h_{ij} \right) \right] + (I_{\geq} e_i) \circ Q^\top \lambda \delta_{i=1} + \Gamma_{ii} e_i + \Gamma^\top e_i \\
&= I_{\geq} e_i \circ \left[ Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i - \sum_{j=i-1}^{K} h_{ij} Q^\top \lambda_j \right] + e_1 q_1^\top \lambda \delta_{i=1} + \Gamma_{ii} e_i + \Gamma^\top e_i \\
&\overset{(23)}{=} I_{\geq} e_i \circ \left[ Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i \right] - I_{\geq} e_i \circ \left( \sum_{j=i-1}^{K} h_{ij} Q^\top \lambda_j \right) + e_1 c \nabla_c \rho \delta_{i=1} + \Gamma_{ii} e_i + \Gamma^\top e_i \\
&= I_{\geq} e_i \circ \left[ Q^\top \nabla_{q_i} \rho + Q^\top A^\top \lambda_i \right] - \left( \sum_{j=i-1}^{K} h_{ij} I_{\geq} e_i \circ Q^\top \lambda_j \right) + e_1 c \nabla_c \rho \delta_{i=1} + \Gamma_{ii} e_i + \Gamma^\top e_i
\end{aligned} \tag{24}$$

We can use (21) to rewrite the term in round parentheses:

$$\begin{aligned}
(21): \quad & (\mathbf{1} - I_{\ll}) \circ Q^\top \Lambda = (\mathbf{1} - I_{\ll}) \circ \nabla_H \rho \\
\implies \quad & (\mathbf{1} - I_{<}) \circ Q^\top \Lambda = (\mathbf{1} - I_{<}) \circ \nabla_H \rho \\
\iff \quad & I_{\geq} \circ Q^\top \Lambda = I_{\geq} \circ \nabla_H \rho \\
\implies \quad & I_{\geq} e_i \circ Q^\top \lambda_i = I_{\geq} e_i \circ \nabla_H \rho e_i
\end{aligned}$$

We plug this into (24), which then yields an expression we can use to solve for a column $i$ of $\Gamma^\top$, requiring knowledge only of the value of $\lambda_i$:

$$\Gamma_{ii}e_i + \Gamma^\top e_i = -I_{\geq}e_i \circ \left[Q^\top \nabla_{q_i}\rho + Q^\top A^\top \lambda_i\right] + \left(\sum_{j=i-1}^{K} h_{ij} I_{\geq}e_i \circ \nabla_H \rho e_i\right) - e_1 c \nabla_c \rho \delta_{i=1} \tag{25}$$

Finally, to solve for $\lambda_i$ and $\lambda$, we look again at (13) and restrict to column $i$:

$$\begin{aligned}
(13) \implies \qquad 0 &= \nabla_{q_i}\rho + A^\top \lambda_i - \Lambda H^\top e_i + \lambda \delta_{i=1} + Q\Gamma e_i + Q\Gamma^\top e_i + r\gamma^\top e_i \\
&= \nabla_{q_i}\rho + A^\top \lambda_i - \sum_{j=i-1}^{K} h_{ij}\lambda_j + \lambda\delta_{i=1} + Q(\Gamma + \Gamma^\top)e_i + r\gamma^\top e_i \\
\iff \begin{matrix} -\lambda & i=1 \\ h_{i,i-1}\lambda_{i-1} & i>1 \end{matrix} \Bigg\} &= \nabla_{q_i}\rho + A^\top \lambda_i - \sum_{j=i}^{K} h_{ij}\lambda_j + Q(\Gamma + \Gamma^\top)e_i + r\gamma^\top e_i
\end{aligned} \tag{26}$$

We now have all equations necessary to solve for all adjoint variables. There is a part that we have not yet addressed: In (26), we will need access to $(\Gamma + \Gamma^\top)e_i$, however (25) will only give us $\Gamma^\top e_i$. How we do this is best explained with an example.

**Example 1** (Construction of $\Gamma + \Gamma^\top$)**.** Let $K = 4$. For decreasing $i$ starting from $K$ down to 1, the known entries in upper triangular $\Gamma^\top \in \mathbb{R}^{4\times 4}$ will evolve as follows, where ? denotes unknown entries and $*$ denotes known entries:

$$\begin{array}{cccccc} & i=4 & & i=3 & & i=2 & & i=1 \\ \Gamma^\top: & \begin{bmatrix} ? & ? & ? & * \\ 0 & ? & ? & * \\ 0 & 0 & ? & * \\ 0 & 0 & 0 & * \end{bmatrix} & & \begin{bmatrix} ? & ? & * & * \\ 0 & ? & * & * \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \end{bmatrix} & & \begin{bmatrix} ? & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \end{bmatrix} & & \begin{bmatrix} * & * & * & * \\ 0 & * & * & * \\ 0 & 0 & * & * \\ 0 & 0 & 0 & * \end{bmatrix} \end{array}$$

This is enough information to then also determine the $i$th column of symmetric $\Gamma + \Gamma^\top$, whose entries we will uncover in the following sequence:

$$\begin{array}{cccccc} & i=4 & & i=3 & & i=2 & & i=1 \\ \Gamma+\Gamma^\top: & \begin{bmatrix} ? & ? & ? & * \\ ? & ? & ? & * \\ ? & ? & ? & * \\ * & * & * & * \end{bmatrix} & & \begin{bmatrix} ? & ? & * & * \\ ? & ? & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} & & \begin{bmatrix} ? & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} & & \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \end{array}$$

We now give the order in which to apply all the equations:
Initially, use (22) to get $\gamma$, then follow that up with (19) to get $\lambda_K$.
Loop for $i = K$ down to 1:
Use (25) to get $\Gamma^\top e_i$ to then get $(\Gamma + \Gamma^\top)e_i$. Use in (26) to solve for $\lambda_{i-1}$, or, if $i = 1$, $\lambda$.

## 3.3 Derivation of the Stable Adjoint System

As described in 2.5, we can use $\overline{\texttt{Bidiagonalize}}_K$ to (inefficiently) bidiagonalize any matrix $A$ with initial vector $v$. The detour through `Hessenberg` let's us use the adjoint system of Hessenberg Factorization described in 3.1. In this section we will inspect the adjoint system of $\overline{\texttt{Bidiagonalize}}_K$, and simplify it considerably by exploiting sparsity in $\dddot{A}$, $\ddot{v}$, $\dddot{Q}$, $\dddot{H}$ and $\ddot{r}$.

The sparsity patterns of $\nabla_{\dddot{Q}}\rho$, $\nabla_{\dddot{H}}\rho$, and $\nabla_{\ddot{r}}\rho$ match those of $\dddot{Q}$, $\dddot{H}$, $\ddot{r}$, due to the linear nature of `extract`:

$$\nabla_{\dddot{Q}}\rho = \begin{bmatrix} \mathbf{0} & \nabla_{l_1}\rho & \mathbf{0} & \nabla_{l_2}\rho & \cdots & \mathbf{0} & \nabla_{l_k}\rho \\ \nabla_{r_1}\rho & \mathbf{0} & \nabla_{r_2}\rho & \mathbf{0} & \cdots & \nabla_{r_k}\rho & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{(N+M)\times K} \qquad \nabla_{\ddot{r}}\rho = \begin{pmatrix} \mathbf{0} \\ \nabla_r\rho \end{pmatrix} \in \mathbb{R}^{N+M}$$

$$\nabla_{\dddot{H}}\rho = \begin{bmatrix} 0 & \nabla_{\alpha_1}\rho & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \nabla_{\beta_1}\rho & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \nabla_{\alpha_2}\rho & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & \nabla_{\beta_2}\rho & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & \ddots & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & \nabla_{\beta_{K-1}}\rho & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \nabla_{\alpha_K}\rho \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \in \mathbb{R}^{2K\times 2K}$$

We will look at the constraints (13) through (16) and determine further sparsity patterns in the adjoint variables $\Lambda, \Gamma, \lambda, \gamma$. We will mark possibly nonzero entries with $*$, and constant zero entries with a 0.

Starting from (22), replacing $K$ with $2K$, we plug in our sparse gradients and outputs, and show that, for this input, $\gamma \in \mathbb{R}^{2K}$ is alternatingly non-zero and zero.

$$(22) \implies \gamma = \nabla_{\dddot{H}}\rho e_{2K} - \dddot{Q}^\top \nabla_{\ddot{r}}\rho = \nabla_{\alpha_K}\rho e_{2K-1} - \dddot{Q}^\top \begin{pmatrix} \mathbf{0} \\ \nabla_r\rho \end{pmatrix} \tag{27}$$

$$= \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ \nabla_{\alpha_K}\rho \\ 0 \end{pmatrix} - \begin{pmatrix} r_1^\top \\ \mathbf{0}^\top \\ r_2^\top \\ \mathbf{0}^\top \\ \vdots \\ \mathbf{0}^\top \\ r_K^\top \\ \mathbf{0}^\top \end{pmatrix} \nabla_r\rho \equiv \begin{pmatrix} * \\ 0 \\ * \\ 0 \\ \vdots \\ 0 \\ * \\ 0 \end{pmatrix} \tag{28}$$

We plug this pattern into (19), which shows that $\lambda_2 K$ has the same sparsity pattern as $\dddot{v}$:

$$(19) \implies \lambda_2 K = \nabla_{\dddot{r}\cdot\cdot}\rho + \dddot{Q}\gamma \equiv \begin{pmatrix} \mathbf{0} \\ \nabla_r \rho \end{pmatrix} + \dddot{Q} \begin{pmatrix} * \\ 0 \\ * \\ 0 \\ \vdots \\ 0 \\ * \\ 0 \end{pmatrix} \equiv \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ * \\ * \\ \vdots \\ * \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ * \\ * \\ \vdots \\ * \end{pmatrix} \equiv \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ * \\ * \\ \vdots \\ * \end{pmatrix} \tag{29}$$

### 3.3.1 Step 3: Profit

We will now give new names to all nonzero entries in $\Lambda, \Gamma, \lambda, \gamma$, and then show a more compact expression for the stable adjoint system. We can now refer to the columns of sparse $\Lambda$ as

$$\Lambda = \begin{bmatrix} \varphi_1 & \mathbf{0} & \varphi_2 & \mathbf{0} & \cdots & \varphi_k & \mathbf{0} \\ \mathbf{0} & \xi_1 & \mathbf{0} & \xi_2 & \cdots & \mathbf{0} & \xi_k \end{bmatrix} \in \mathbb{R}^{(N+M)\times 2K}$$

As shown, $\Gamma$ has a checkered pattern, which, here shown for $K = 6$ we name as:

$$\Gamma = \begin{bmatrix} \omega_{11} & 0 & 0 & 0 & 0 & 0 \\ 0 & \sigma_{11} & 0 & 0 & 0 & 0 \\ \omega_{21} & 0 & \omega_{22} & 0 & 0 & 0 \\ 0 & \sigma_{21} & 0 & \sigma_{22} & 0 & 0 \\ \omega_{31} & 0 & \omega_{32} & 0 & \omega_{33} & 0 \\ 0 & \sigma_{31} & 0 & \sigma_{32} & 0 & \sigma_{33} \end{bmatrix} \quad \text{with aliases} \quad \Omega = \begin{bmatrix} \omega_{11} & 0 & 0 \\ \omega_{21} & \omega_{22} & 0 \\ \omega_{31} & \omega_{32} & \omega_{33} \end{bmatrix}$$

$$\Sigma = \begin{bmatrix} \sigma_{11} & 0 & 0 \\ \sigma_{21} & \sigma_{22} & 0 \\ \sigma_{31} & \sigma_{32} & \sigma_{33} \end{bmatrix}$$

and the entries of $\lambda$ and $\gamma$ as

$$\lambda = \begin{bmatrix} \mathbf{0} \\ \xi_0 \end{bmatrix} \qquad \gamma = \begin{bmatrix} \gamma_1 & 0 & \gamma_2 & 0 & \cdots & 0 & \gamma_K & 0 \end{bmatrix}^\top$$

## 3.4 Columnwise expanded unstable adjoint

Define $b_n\lambda_{n+1} := \mathbf{0}$ and $b_0\rho_0 := \mathbf{0}$

$$0 = \nabla l_n + a_n\lambda_n + b_n\lambda_{n+1} - A\rho_n + l_n\sigma_n \tag{30}$$
$$0 = \nabla r_n - A^\top\lambda_n + b_{n-1}\rho_{n-1} + a_n\rho_n + r_n\omega_n + \delta_{i,1}\kappa \tag{31}$$
$$\rho_k = -\nabla res \tag{32}$$
$$\nabla a_n + l_n^\top\lambda_n + r_n^\top\rho_n = 0 \text{ for } i \in \{1\cdots k\} \tag{33}$$
$$\nabla b_n + l_n^\top\lambda_{n+1} + r_{n+1}^\top\rho_n = 0 \text{ for } i \in \{1\cdots k-1\} \tag{34}$$
$$\nabla c - \tilde{r}^\top\kappa = 0 \tag{35}$$
$$\nabla\tilde{r} = -c\kappa \tag{36}$$
$$\nabla A = \sum_{n=1}^{k} -\lambda_n r_n^\top - l_n\rho_n^\top \tag{37}$$

## 3.5 Hessenberg factorization has stable adjoint because of reprojection

Some of the constraints of the adjoint system in Hessenberg factorization allow for a direct correction of drift [4]. Because Hessenberg factorization subsumes Bidiagonalization for this specific structured input, we can use their implementation of numerically stable gradients to differentiate bidiagonalization.

## 3.6 Unstable Primal constraints

Define $l_0b_0 := 0$ and $res := r_{k+1}b_k$

$$\begin{aligned} r_1 &= c\tilde{r} \\ \alpha_n l_n &= Ar_n - \beta_{n-1}l_{n-1} \\ \beta_n r_{n+1} &= A^\top l_n - \alpha_n r_n \\ \langle l_i, l_i\rangle &= 1 \text{ for } i \in 1\ldots k \\ \langle r_i, r_i\rangle &= 1 \text{ for } i \in 1\ldots k \end{aligned}$$

Symbolically, we can[5] show that these constraints, if satisfied, imply $\langle l_i, l_j\rangle = \delta_{i,j}$ and $\langle r_i, r_j\rangle = \delta_{i,j}$. In practice however, we need to reorthonormalize the produced vectors during iteration.
These "classic" forward constraints and the associated adjoint system:

[4] cite equation 13b
[5] proof by induction. Add?

## 3.7 Unstable Adjoint system

[6] This system can be derived from the bidiagonalization forward constraints directly.

$$\rho_k = -\nabla res \tag{38}$$
$$c\nabla c = r_1^\top \kappa \tag{39}$$
$$\nabla \tilde{r} = -c\kappa \tag{40}$$
$$0 = \nabla L - AP + \Lambda B^\top + L\Sigma \tag{41}$$
$$0 = \nabla R - A^\top \Lambda + PB + R\Omega + \delta_{i,1}\kappa \tag{42}$$
$$\langle l_n, \lambda_n \rangle + \langle r_n, \rho_n \rangle = \nabla a_n \text{ for } i \in \{1 \cdots k\} \tag{43}$$
$$\langle l_n, \lambda_{n+1} \rangle + \langle r_{n+1}, \rho_n \rangle = \nabla b_n \text{ for } i \in \{1 \cdots k-1\} \tag{44}$$
$$\nabla A = \sum_{n=1}^{k} -\lambda_n r_n^\top - l_n \rho_n^\top \tag{45}$$

## 3.8 Stable Adjoint System

$$\downarrow_k = \nabla_{res} + R\gamma \tag{46}$$
$$r_1^\top \kappa = c\nabla_c \tag{47}$$
$$0 = \nabla_L + A \Downarrow - \Uparrow B^\top + L(\Sigma + \Sigma^\top) \tag{48}$$
$$0 = \nabla_R + A^\top \Uparrow - \Downarrow B + R(\Omega + \Omega^\top) + \kappa e_1^\top + res\gamma^\top \tag{49}$$
$$\langle l_i, \uparrow_j \rangle = \nabla \beta_{i-1} e_{i-1} \text{ for } i \leq j \tag{50}$$
$$\langle r_i, \downarrow_j \rangle = \nabla \alpha_i e_i \text{ for } i \leq j+1 \tag{51}$$
$$\nabla_v = -\kappa c \tag{52}$$
$$\nabla_A = \sum_{i=1}^{k} l_i \downarrow_i^\top + \uparrow_i r_i^\top \tag{53}$$

## 3.9 The Adjoint Method to compute gradients

The adjoint method provides an elegant way to compute parameter gradients of losses that depend on outputs of parameterized numerical solvers.

Let $s : \theta \mapsto x$ be a numerical solver that yields a solution $x$ to some problem $\theta$, with $\dim(\theta) = P$ and $\dim(x) = X$. Assume also that we know an associated constraint function $c : x \mapsto \mathbb{R}^X$ that evaluates all the constraints of the problem, for which $c(s(\theta), \theta) = \mathbf{0}$ for all $\theta$. Additionally, require that $\frac{\partial c}{\partial \theta}$ is everywhere nonsingular. Assume also that we have a loss function $\ell : x \mapsto \mathbb{R}$ that depends on the solution $x$ of the problem. We are interested in $\nabla_\theta \ell(s(\theta))$, the gradient of $\ell$ with respect to the parameters $\theta$.

[6] See another subsection for the same things all expanded columnwise

**Example 2** (Hessenberg Factorization)**.** A fitting example would be $s = \texttt{Hessenberg}$, which implies that $\theta = (A, v)$ and $x = (Q, H, r, c)$. The associated constraint function would be the concatenation of constraints (8) through (12). We see that, for this example, $c(x, \theta) = \mathbf{0}$ for all solutions $(Q, H, r, c)$ if and only if they correctly Hessenberg-factorize from $\theta = (A, v)$.

Because $c(s(\theta), \theta) = \mathbf{0}$ is constantly zero, the total derivative $\frac{d}{d\theta}c$ is also zero. With the chain rule, we have:

$$\mathbf{0} = \frac{d}{d\theta}c(s(\theta), \theta) = \frac{\partial c}{\partial s}\frac{\partial s}{\partial \theta} + \frac{\partial c}{\partial \theta} \implies \frac{\partial c}{\partial s}\frac{\partial s}{\partial \theta} = -\frac{\partial c}{\partial \theta} \tag{54}$$

The adjoint method will exploit this relationship. We apply the chain rule to compute the gradient $\nabla_\theta \ell(s(\theta))$:

$$\nabla_\theta \ell(s(\theta)) = \frac{\partial s}{\partial \theta}^\top \nabla_x \ell(s(\theta)) \tag{55}$$

We assume that $\nabla_x \ell(s(\theta))$ is easy to obtain. The challenge is computing $\frac{\partial s}{\partial \theta} \in \mathbb{R}^{X \times P}$, which can be expensive for high-dimensional $x$. Suppose we can find *adjoint variables* $\boldsymbol{\lambda} \in \mathbb{R}^X$ solving the uniquely determined $X \times X$ linear system (unique by initial assumption on the constraint Jacobian)

$$\nabla_x \ell(s(\theta)) = \frac{\partial c}{\partial s}^\top \boldsymbol{\lambda} \tag{56}$$

Then, substituting this into (55) yields:

$$\nabla_\theta \ell(s(\theta)) = \frac{\partial s}{\partial \theta}^\top \frac{\partial c}{\partial s}^\top \boldsymbol{\lambda} = \left(\frac{\partial c}{\partial s}\frac{\partial s}{\partial \theta}\right)^\top \boldsymbol{\lambda} \overset{(54)}{=} -\frac{\partial c}{\partial \theta}^\top \boldsymbol{\lambda}$$

The adjoint method provides significant computational savings compared to the standard approach. To see this, consider the two approaches:

*Forward approach:* Compute $\frac{\partial s}{\partial \theta} \in \mathbb{R}^{X \times P}$ directly by solving $P$ linear systems of size $X \times X$ (one for each parameter), then compute the gradient via (55). Total cost: $O(PX^3)$ operations.

*Adjoint approach:* Solve one linear system (56) of size $X \times X$ to find $\boldsymbol{\lambda}$, then evaluate $-\frac{\partial c}{\partial \theta}^\top \boldsymbol{\lambda}$. Total cost: $O(X^3 + XP)$ operations.

When $P \ll X$ (few parameters, high-dimensional state), the adjoint method is dramatically more efficient: $O(X^3)$ versus $O(PX^3)$. This advantage becomes crucial in applications like neural network training or PDE-constrained optimization where $X$ can be very large.

# 4 Numerical Stability and Experiments

## 4.1 Reprojection and Reorthonormalization

Let $m$ be a mask vector with entries from $\{0, 1\}$. Let $L$ be an orthonormal matrix, and $q$ and $c$ be vectors. Consider a constraint of the form

$$m \circ (L^\top q) = m \circ c$$

Paraphrased: "Within the bounds of the mask $m$, we have that $q$ expressed in the basis of $L$ equals $c$". If we have access to a numerical estimate $q_0$ of a solution $q$, we can get an improved estimate $q_1$ by *reprojecting* it against $L$.

$$\begin{aligned} q_0 &= q_0 - L(m \circ L^\top q_0) + L(m \circ L^\top q_0) \\ &= q_0 - L(m \circ L^\top q_0) + L(m \circ c) \end{aligned}$$

The following motivates following update step, which can be repeated if necessary.

$$q_1 \leftarrow q_0 - L(m \circ L^\top q_0) + L(m \circ c)$$

In the case of Hessenberg factorization, we have the constraint $Q^\top q_i = e_i$ (10) in which case we call the reprojection step followed by normalization *reorthonormalization*.
For Bidiagonalization, we have similar constraints $L^\top l_i = e_i$ (4) and $R^\top r_i = e_i$ (5). Empirically, we find that two repetitions of the reprojection step are necessary to eliminate numerical errors. TODO show this on a Hilbert matrix?